

Protocol Zone

*A project report submitted to ICT Academy of Kerala
in partial fulfillment of the requirements
for the certification of*

Data Science

&

Analytics

submitted by

ALAINA S RAJ

ARCHA S A

ABHIMANYU V

ALEN SONY

GOPIKA G



ICT ACADEMY OF KERALA
THIRUVANANTHAPURAM, KERALA, INDIA

May 2026

List of Figures

- 1) Flow chart
- 2) Use case diagram
- 3) Data base structure
- 4) Data flow diagrams

Figure 1: Protocol Zone Web App Flowchart

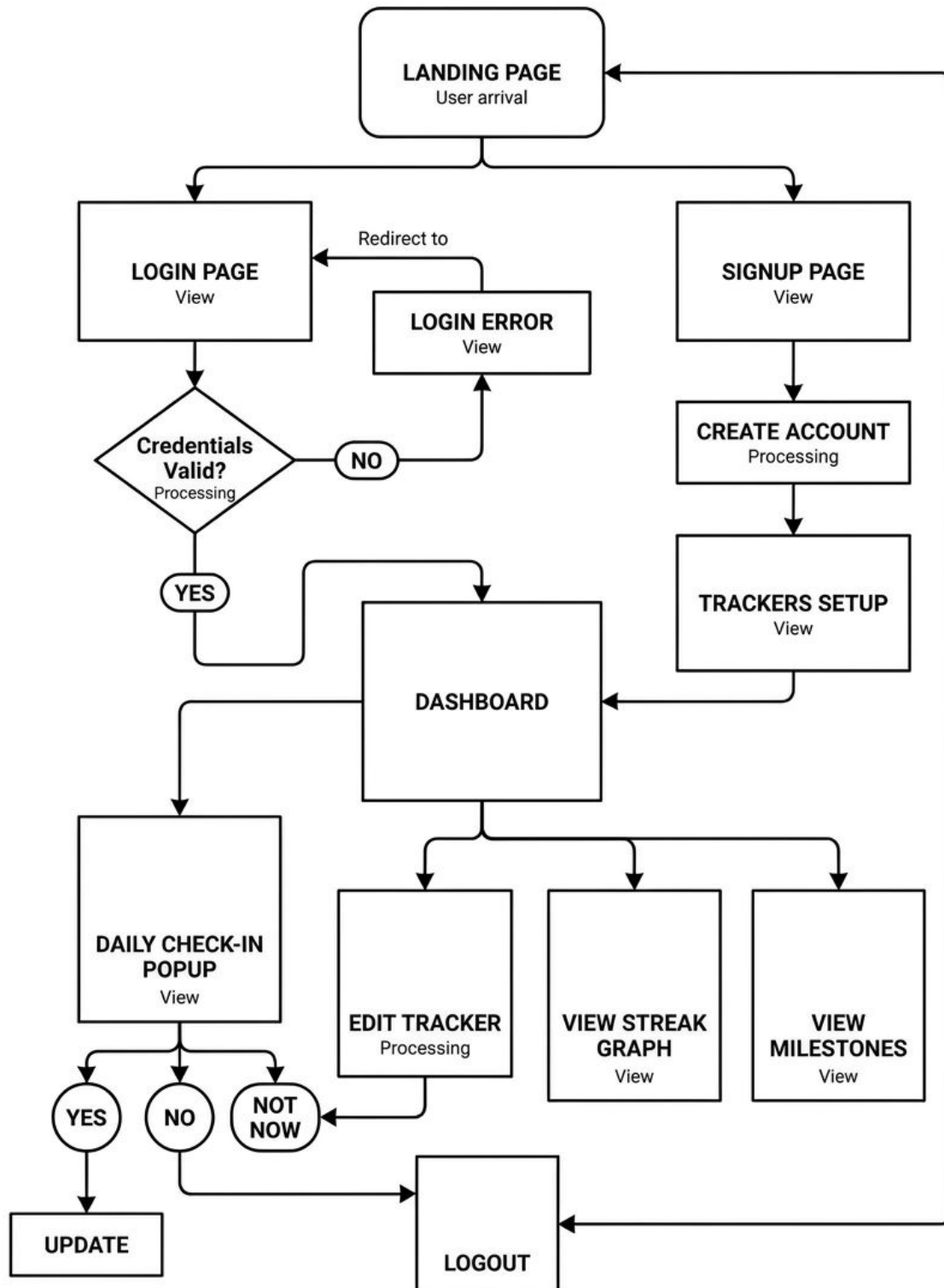


Figure 2: Protocol Zone Use Case Diagram

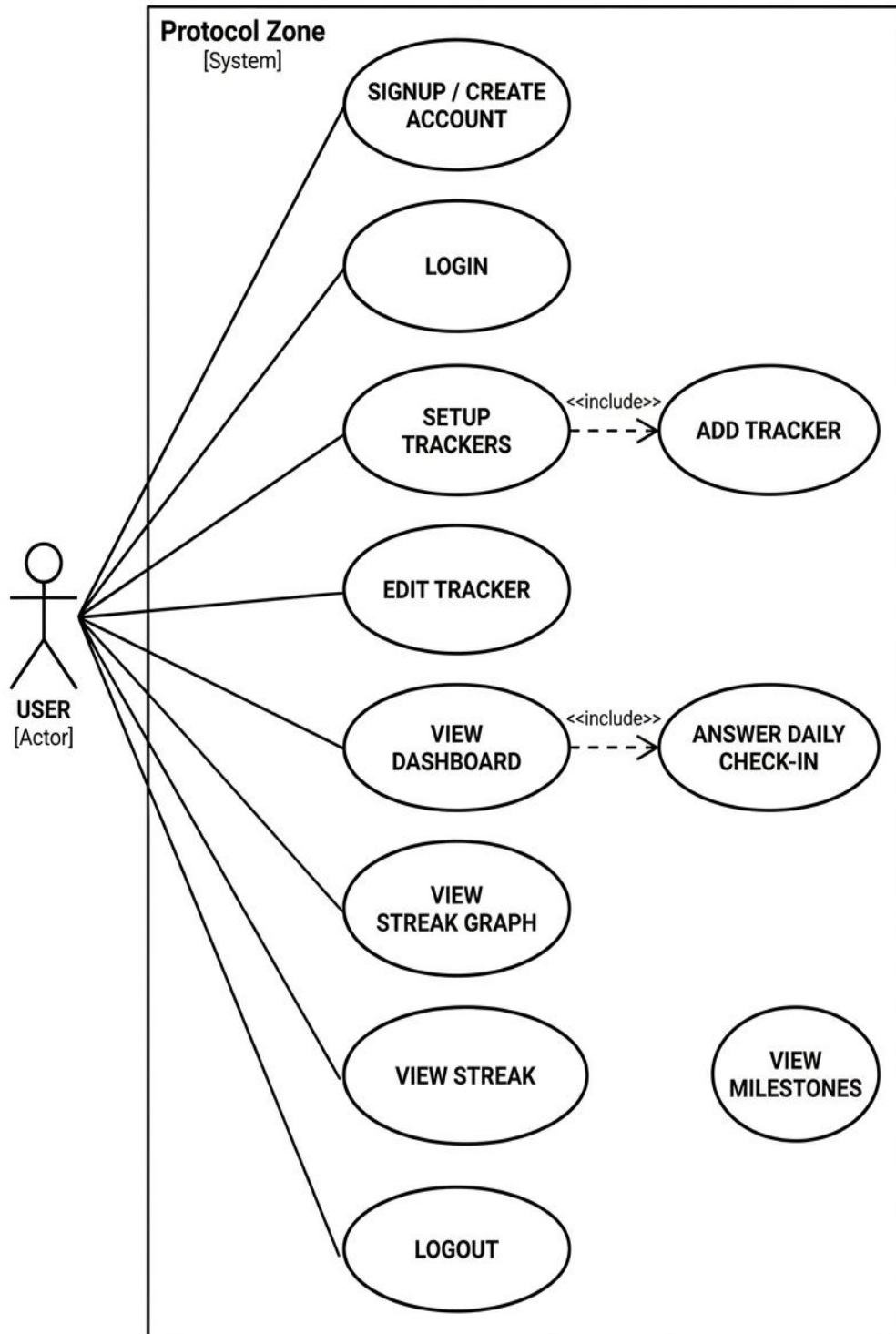
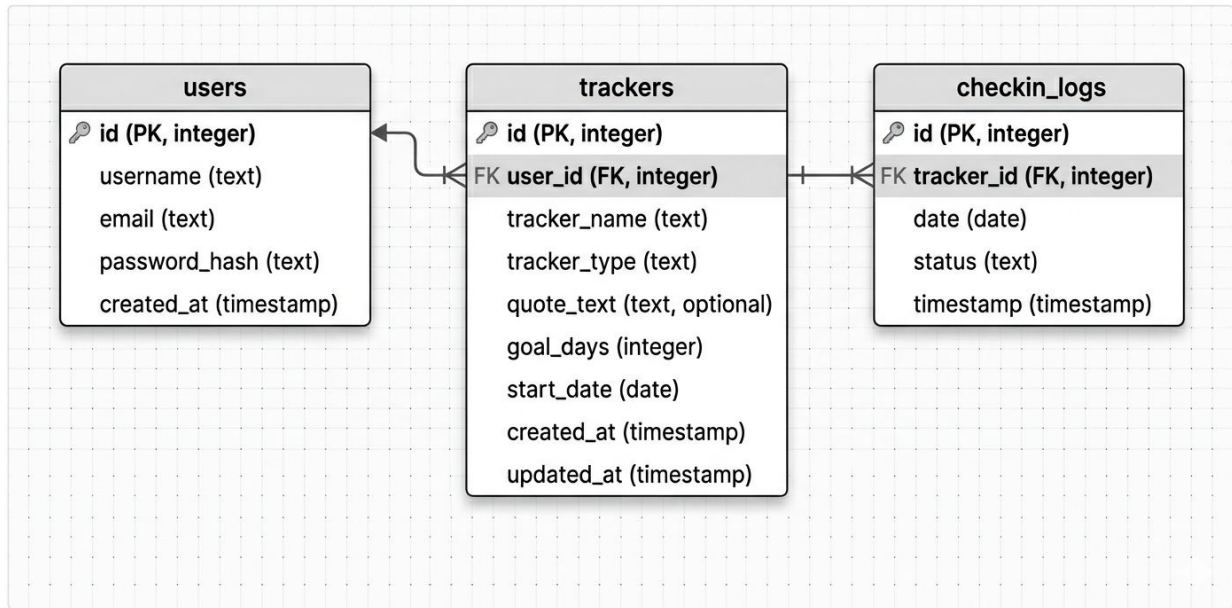
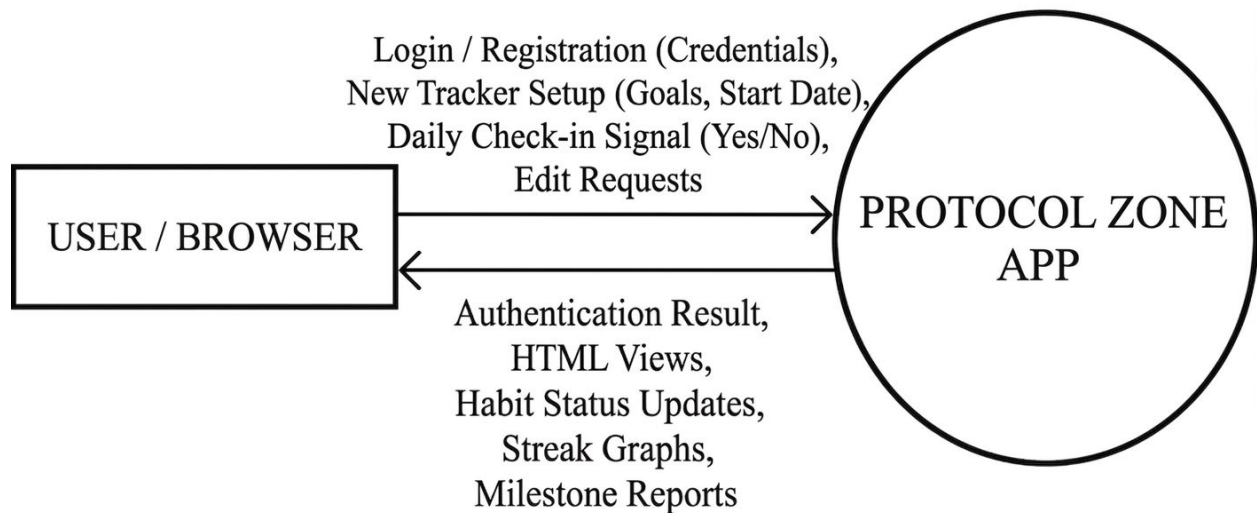


Figure 3: Database table structure for Protocol Zone App

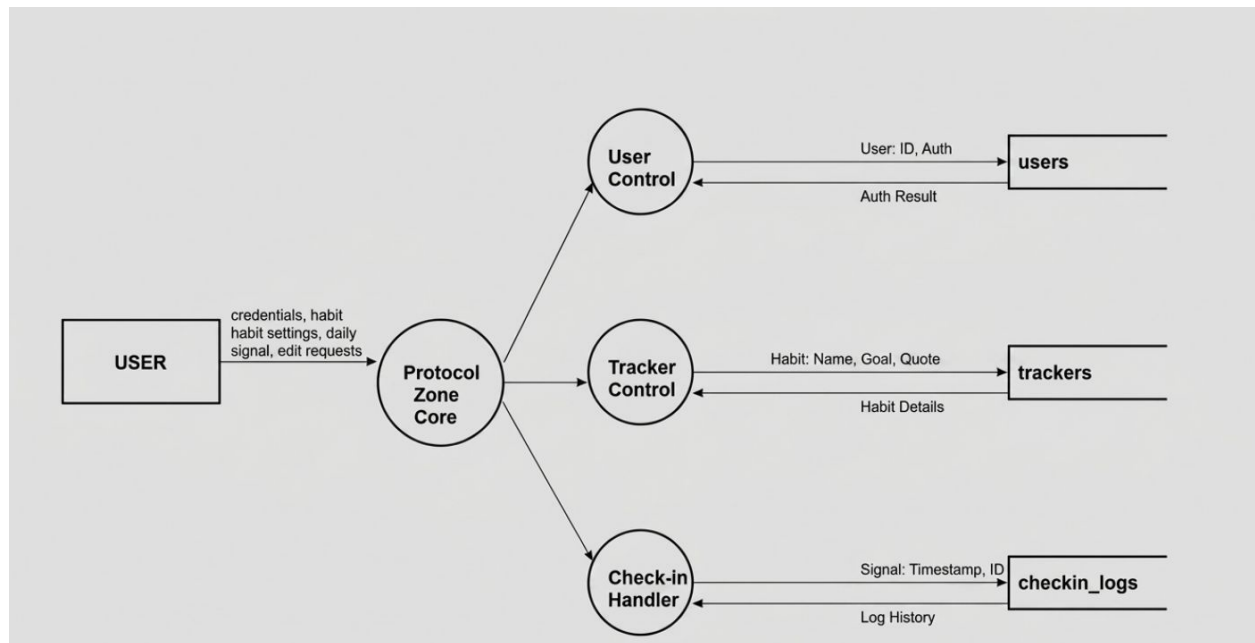


Data flow diagrams

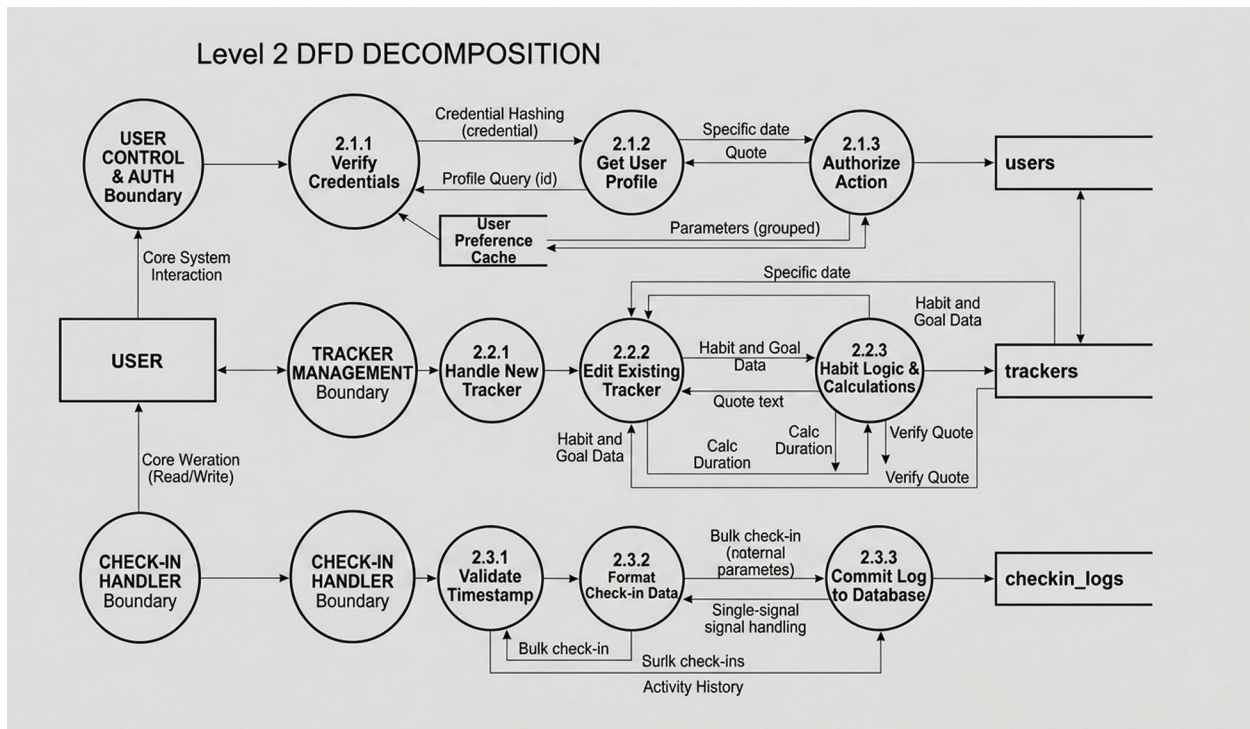
Level 0 DFD - Protocol Zone Context Diagram



Level 1 context



Level 2 Decomposition



List of Abbreviations

CSS - Cascading Style Sheets

DB - Database

DFD - Data Flow Diagram

HTML - HyperText Markup Language

SQLite - Lightweight relational database used by the application

UI - User Interface

Table of Contents

Abstract

1. Problem Definition

1.1 Overview

1.2 Problem Statement

2. Introduction

3. Literature Survey

4. Design

5. Coding / Implementation

6. Deployment

7. Result

8. Conclusion

9. References

Abstract

Protocol Zone is a beginner-friendly Flask web application developed to help users track habits, goals, and study subjects. The system allows users to sign up, log in, create up to three trackers, answer daily check-ins, view calendar-based progress, and understand consistency using streaks.

The application uses Python and Flask for backend logic, SQLite for persistent storage, HTML and CSS for the user interface, and Matplotlib for generating a streak graph. A milestone feature motivates users by showing progress levels based on the current streak.

1. Problem Definition

This section describes the purpose of the application and the problem it solves for users who want simple progress tracking.

1.1 Overview

Protocol Zone focuses on personal progress tracking. A user can create trackers for habits, goals, or study subjects. Each tracker stores a name, type, quote, goal days, and start date. The dashboard displays the current streak, quote, calendar, graph link, and milestone link.

The project intentionally uses a simple Flask and SQLite structure so the code remains understandable for beginner-level development. The main Python file handles routes and app logic, while the database module handles table creation and data access.

1.2 Problem Statement

Many users lose consistency because they cannot clearly see daily progress or recover missed check-ins. A simple tracker should show what was completed, what was missed, and what level of consistency has been achieved without using a complex interface.

Protocol Zone solves this by combining daily check-ins, calendar colors, streak counting, graph visualization, and milestone levels in one lightweight web application.

2. Introduction

The application starts with signup and login. After authentication, the user can create trackers from three types: subject, habit, and goal. Each tracker has a goal day count and an optional motivational quote.

The dashboard is the main screen. It shows the selected tracker, current streak, goal days, quote, calendar, and navigation links for the streak graph and milestones. The calendar allows users to click valid dates and answer the check-in popup with YES, NO, or NOT NOW.

The check-in logic saves YES as clean and NO as slipped. NOT NOW only snoozes the popup and does not save progress. The app uses Flask sessions to remember the logged-in user and popup cooldowns.

3. Literature Survey

Flask was selected because it is lightweight and suitable for beginner-friendly web applications. Its route-based structure makes it easy to connect URLs to Python functions and HTML templates.

SQLite was selected because it does not require a separate database server. It stores users, trackers, and check-in logs in a single local database file.

Matplotlib was used to create the streak graph. The graph is generated as a PNG image in memory using BytesIO and returned through a Flask response. This avoids saving temporary image files on disk.

PythonAnywhere was used for deployment because it supports Flask applications, virtual environments, Git-based updates, and simple web app reloading.

4. Design

This section outlines the architectural design of Protocol Zone, including the overall system structure, data flow, user interface, and database schema, ensuring the application supports efficient data collection, processing, and visualization for habit tracking analytics.

System Architecture

The application follows a three-tier architecture: the presentation layer (HTML/CSS templates for user interaction), the application layer (Flask routes handling logic and data processing), and the data layer (SQLite database for persistent storage), which allows for scalable data analytics without external dependencies.

User Interface Design

The UI employs a simple, responsive design with Bootstrap-inspired CSS for mobile-friendly layouts, featuring a dashboard calendar that color-codes days (green for clean, red for slipped) to provide intuitive visual analytics of progress patterns.

Forms for signup, login, and tracker setup include validation feedback to ensure data integrity, while popup modals for check-ins minimize disruption and support quick data entry for daily tracking.

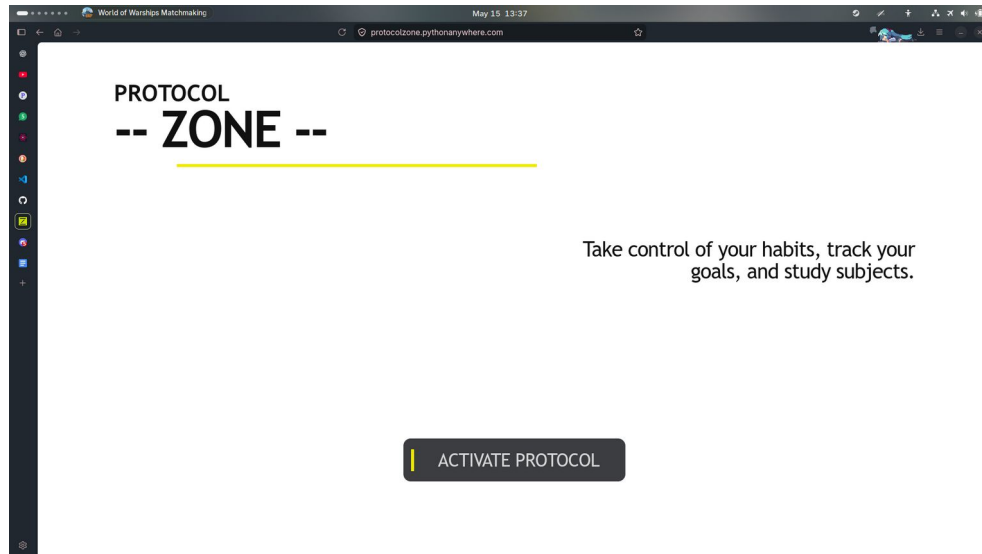
Database Design

The relational database schema consists of three core tables: users for authentication data, trackers for habit metadata, and checkin_logs for time-series check-in statuses, linked by foreign keys to enable efficient queries for streak analytics and milestone calculations.

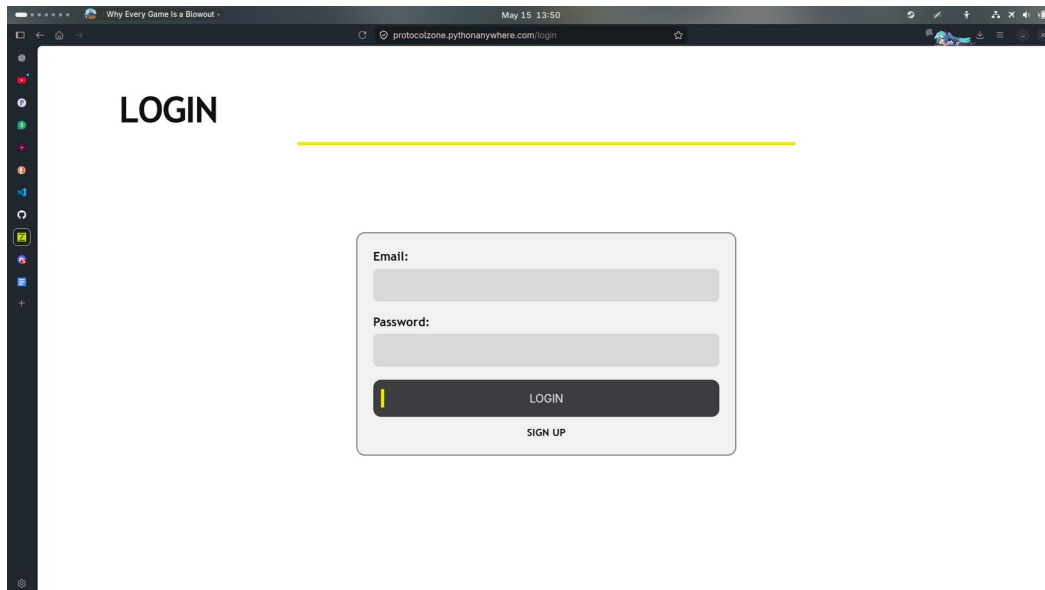
Data normalization ensures minimal redundancy, with constraints on tracker types and goal days to maintain consistency in analytical computations.

Screenshots

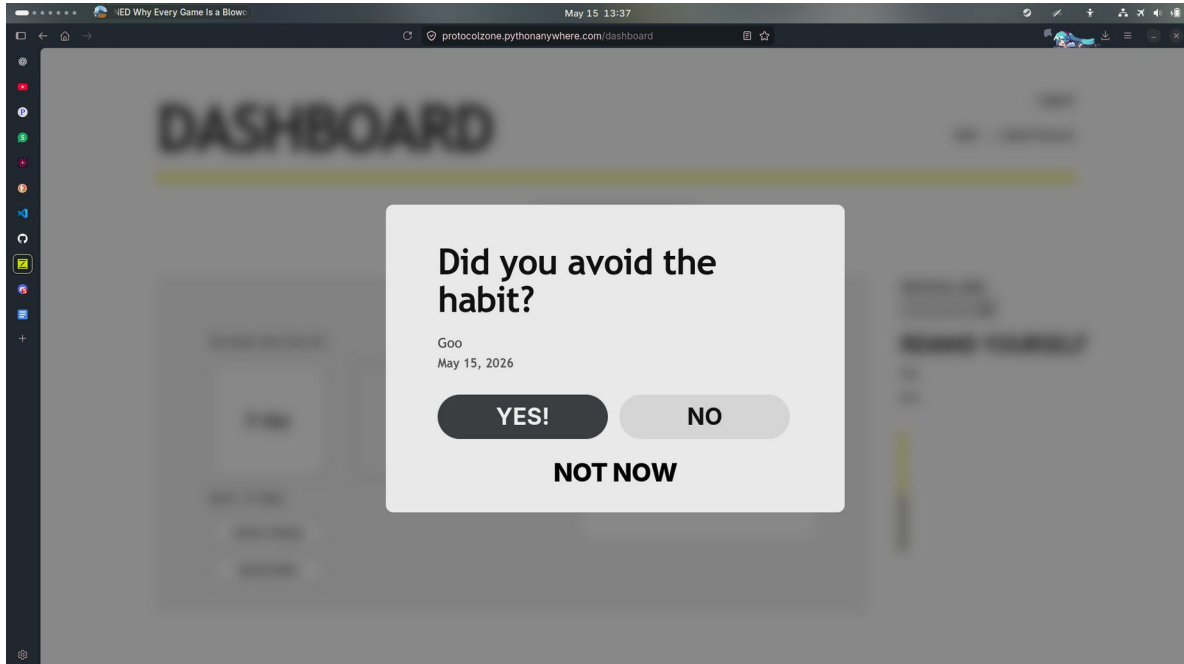
Main screen:



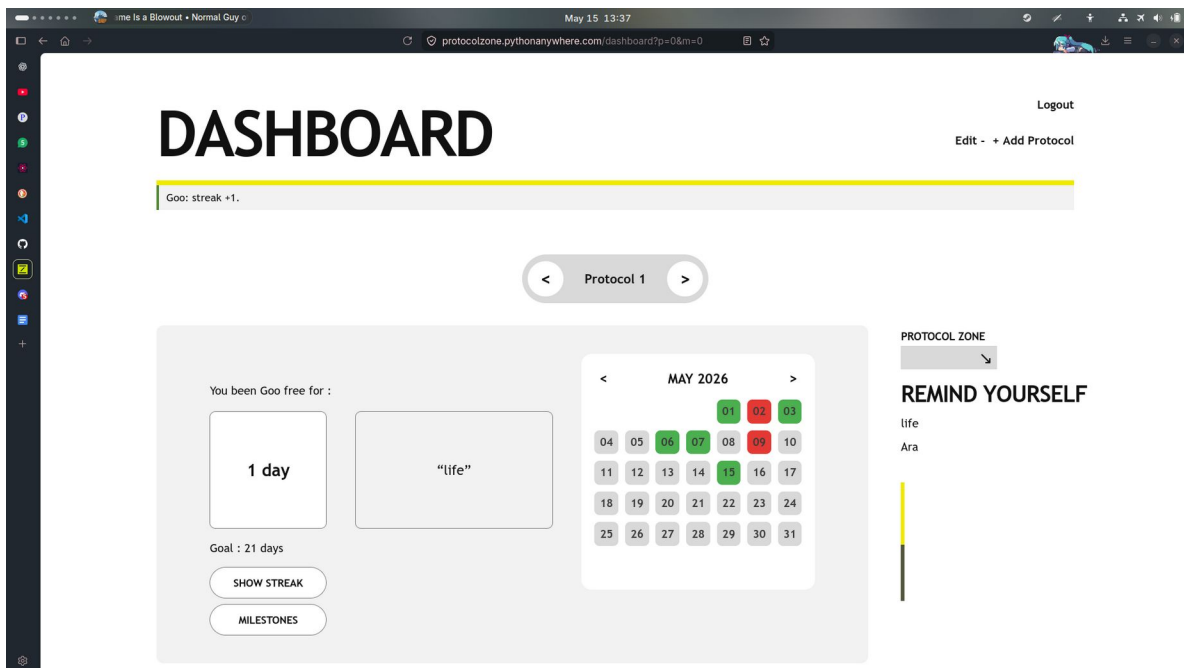
Login Page:



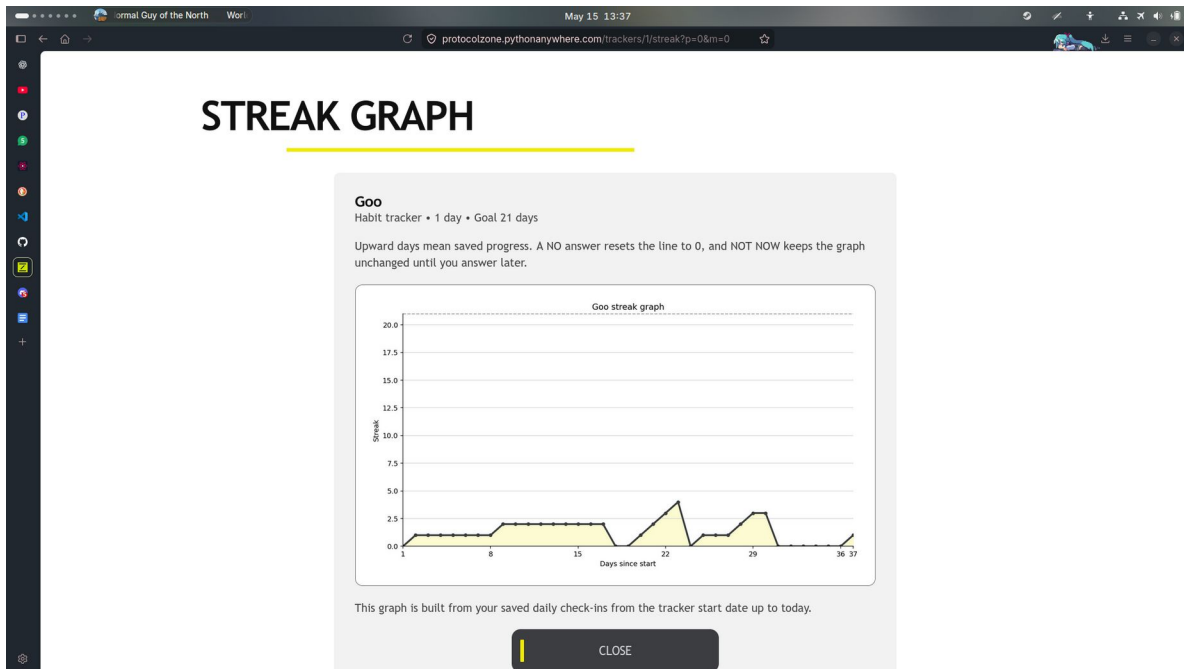
Check-in screen



Dashboard



Streak graph



Milestone screen

The screenshot shows a web browser window displaying the 'MILESTONES' for the 'Goo' habit tracker. The browser's address bar shows the URL 'protocolzone.pythonanywhere.com/trackers/?p=0&'. The page title is 'MILESTONES'. Below the title, the habit tracker details are shown: 'Goo', 'Habit tracker • 1 day • Goal 21 days'. A note explains: 'Milestones are based on your current streak. Keep answering your daily check-ins to unlock the next level.' The main feature is a list of milestones. The first milestone is 'NEXT MILESTONE', 'Getting Warm', '2 days remaining to reach 3 days.', and 'You are building proof that you can return to this.' The second milestone is '1 DAY MILESTONE', 'First Step', 'Unlocked', and 'You started. That is the hardest door to open.' The third milestone is '3 DAY MILESTONE', 'Getting Warm', '2 days left', and 'You are building proof that you can return to this.' The fourth milestone is '7 DAY MILESTONE', 'Getting Warm', and '2 days left'. The milestones are displayed in a list with a yellow background for the 3-day milestone and a dark grey background for the 7-day milestone.

Add/ Edit tracker

The screenshot shows a web browser window with the URL `protocolzone.pythonanywhere.com/trackers/setup`. The page title is "ADD TRACKER". On the left is a dark sidebar with various icons. The main content area is divided into two sections: "New Tracker" and "Existing Trackers".

New Tracker
1 / 3 trackers added

Name
Math revision, meditation, coding streak

Type
Subject

Goal days (1-254)
21

Quote [your reason to do this task/goal]
Why this matters to you

Buttons: ADD TRACKER, FINISH SETUP

Existing Trackers
These will appear on your dashboard.

Goo
Habit • 21 days
life

Edit tracker

The screenshot shows a web browser window with the URL `protocolzone.pythonanywhere.com/trackers/?p=0&m=0`. The page title is "EDIT TRACKER". On the left is a dark sidebar with various icons. The main content area contains a form for editing a tracker named "Goo".

Goo
Habit tracker

Name
Goo

Goal days (1-254)
21

Quote [your reason to do this task/goal]
life

If the new goal days are smaller than your current tracked days, this tracker resets from today so the counter and calendar stay correct.

Buttons: SAVE CHANGES, CANCEL

5. Coding / Implementation

This section details the key implementation aspects of Protocol Zone, including core Flask routes for user interaction, database operations for data persistence, and logic for streak calculations, highlighting how the application processes and analyzes habit-tracking data.

Flask Routes and Authentication

The login route handles user authentication by validating form inputs, querying the database, and managing sessions, ensuring secure access to personalized data analytics.

```
@app.route('/login', methods=['GET', 'POST']) def login(): """Log an existing user in.""" if get_logged_in_user(): return redirect(url_for('dashboard'))
```

```
# A normal page visit just shows the form.
```

```
if request.method != 'POST':
```

```
    return render_template('login.html')
```

```
# Read the login form fields.
```

```
email = request.form.get('email', '').strip().lower()
```

```
password = request.form.get('password', '').strip()
```

```
# Both fields are required.
```

```
if not email or not password:
```

```
    flash('Please enter your email and password.', 'error')
```

```
    return render_template('login.html')
```

```
# Load the user and compare the entered password with the saved hash.
```

```
user = db.get_user_by_email(email)
```

```
if user is None or not password_matches(password, user['password_hash']):
```

```
    flash('Invalid credentials. Please try again.', 'error')
```

```
    return render_template('login.html')
```

```
# Save the user id in the session after a successful login.
```

```
session['user_id'] = user['id']
```

```
# Brand-new users with no trackers go to setup first.
```

```
if db.get_tracker_count(user['id']) == 0:
```

```
    return redirect(url_for('trackers_setup'))
```

```
return redirect(url_for('dashboard'))
```

Dashboard Logic and Data Processing

The dashboard route aggregates tracker data, processes calendar offsets, and determines check-in prompts, enabling real-time visualization of progress data for analytical insights.

```
@app.route('/dashboard') def dashboard(): """Main dashboard page.""" user = get_logged_in_user() if user is None: return redirect(url_for('login'))
```

```
# Read dashboard state from the URL query string.
selected_index_text = request.args.get('p', '0').strip()
month_offset_text = request.args.get('m', '0').strip()
check_tracker_id_text = request.args.get('check_tracker', '').strip()
check_date_text = request.args.get('check_date', '').strip()
```

```
# Convert raw query text into safe Python values.
selected_index = parse_int(selected_index_text, 0)
month_offset = parse_int(month_offset_text, 0)
requested_tracker_id = parse_int(check_tracker_id_text)
requested_check_date = parse_iso_date(check_date_text)
```

```
# Load and prepare every tracker for dashboard display.
trackers = load_dashboard_trackers(user['id'], month_offset)
# Keep the selected index inside the valid range.
selected_index = clamp_selected_index(trackers, selected_index)
active_tracker = trackers[selected_index] if trackers else None
```

```
# Decide whether the yes/no popup should be shown right now.
checkin_prompt = choose_checkin_prompt(
    trackers=trackers,
    selected_index=selected_index,
    month_offset=month_offset,
    requested_tracker_id=requested_tracker_id,
    requested_check_date=requested_check_date,
)
```

```
return render_template(
    'dashboard.html',
    user=user,
    trackers=trackers,
    active_tracker=active_tracker,
    selected_index=selected_index,
    month_offset=month_offset,
    total_trackers=len(trackers),
    max_trackers=MAX_TRACKERS,
    tracker_types=TRACKER_TYPES,
    checkin_prompt=checkin_prompt,
)
```

Database Operations

Database functions like user retrieval support efficient data querying for analytics, such as streak computations based on check-in logs stored in SQLite.

```
def get_user_by_email(email): connection = open_database() try: return connection.execute( 'SELECT id,
username, email, password_hash FROM users WHERE email
= ?', (email,), ).fetchone() finally: connection.close()
```

Challenges and Solutions

During implementation, handling session-based cooldowns for check-in popups required careful timestamp logic to prevent user frustration, while Matplotlib graph generation involved optimizing memory usage for dynamic data visualization.

6. Deployment

Protocol Zone was deployed on PythonAnywhere using the platform's Bash console and a dedicated virtual environment to isolate dependencies. The deployment process was:

- Clone or pull the latest code from the Git repository using PythonAnywhere Bash.
- Create a Python virtual environment inside the project folder.
- Activate the virtual environment and install required packages from requirements.txt.
- Configure the PythonAnywhere web app to use the app's WSGI entry point and the virtualenv path.
- Reload the web app after each Git update

A typical deployment workflow on PythonAnywhere Bash looks like this:

```
cd ~/myproject  
git pull origin main  
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

After this setup, the web app was configured in PythonAnywhere to use the virtual environment `.venv` and the Flask app entry point in `wsgi.py` (or `app.py`), then reloaded from the PythonAnywhere dashboard.

This method ensured that the application ran with the same dependency environment used during development, and allowed easy updates by pulling new code from Git and reloading the app.

7. Result

Major Python modules: app.py contains Flask setup, routes, authentication, form validation, dashboard building, calendar logic, check-in handling, streak graph generation, and milestone calculation. database.py contains SQLite connection setup, table creation, migration logic, and CRUD helper functions.

Login logic: the login route reads email and password from the form, validates both fields, finds the user by email, compares the entered password hash with the saved hash, stores user_id in the session, and redirects the user to setup or dashboard.

Tracker logic: the tracker setup route allows a logged-in user to create up to three trackers. The shared form helper validates tracker name, tracker type, quote, and goal days before saving the tracker through database.py.

Check-in logic: the dashboard chooses either a clicked calendar date or today as the popup date. YES is stored as clean, NO is stored as slipped, and NOT NOW only saves a temporary cooldown in the session.

Streak graph logic: the graph starts from the tracker start date and builds one point for each day up to today. Clean days increase the streak, slipped days reset it to zero, and unanswered days keep the previous value. Matplotlib draws the graph and Flask returns it as a PNG image.

Milestone logic: milestone levels are calculated from the current streak. Completed milestones are shown as unlocked, the current milestone is highlighted, and upcoming milestones show how many days are left.

Database tables: users(id, username, email, password_hash, created_at); trackers(id, user_id, tracker_name, tracker_type, quote_text, goal_days, start_date, created_at, updated_at); checkin_logs(id, tracker_id, date, status, timestamp). users.id links to trackers.user_id, and trackers.id links to checkin_logs.tracker_id.

Application flowchart: User opens app -> Signup/Login -> Create tracker -> Dashboard -> Calendar or popup -> YES/NO/NOT NOW -> Save check-in -> Update calendar and streak -> View graph or milestones.

Use case: The user can sign up, log in, create trackers, edit trackers, answer check-ins, update missed dates, view streak graph, and view milestones.

Data flow diagram: User input is received by Flask routes in app.py, validated by helper functions, stored or read through database.py, saved in SQLite, converted into dashboard data, and displayed through HTML templates styled by CSS.

8. Conclusion

Protocol Zone successfully demonstrates a simple full-stack Flask application for habit, goal, and subject tracking. The project includes authentication, tracker creation, daily check-ins, calendar feedback, streak visualization, and milestone-based motivation.

The implementation remains beginner-friendly by using direct Flask routes, SQLite helper functions, readable variable names, simple templates, and CSS-based styling. The project can be extended in the future with stronger authentication, reports, reminders, or more detailed analytics.

9. References

Flask Documentation: <https://flask.palletsprojects.com/>

SQLite Documentation: <https://www.sqlite.org/docs.html>

Matplotlib Documentation: <https://matplotlib.org/stable/index.html>

PythonAnywhere Help: <https://help.pythonanywhere.com/>

Python Documentation: <https://docs.python.org/3/>